Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Ders 12 Nisan 2023 Çarşamba günü online yapıldı. 5 Nisan'da ders yapılmadı.
- Ders İngilizce anlatıldı. Bazı kısımlar Türkçe tekrar edildi.
- Şu ana kadar işlenen bölümler: 1, 2, 3 ve 7. Bu derste 4. bölüm (chapter 4) başladı: veri akışlarında madencilik (Mining Data Streams).
- İşlenen konular:
  - Akış verisi (stream data) ve akış veri yönetim sistemi (DSMS).
  - İlişkisel veritabanı ile akış sistemi arasındaki farklar.
  - Anlık sorgular (ad hoc queries) ve sürekli sorgular (standing queries).
  - Uygulamalar: sorgu akışları, tıklama akışları, IP paketleri.
  - Kayan pencere (sliding window) ve pencerede ortalama güncelleme.
  - Bloom filtresi (Bloom filter): ekleme, sorgulama, yanlış pozitif olasılığı.
  - Son k bitteki 1'leri sayma ve DGIM algoritması.
- Dersin sonunda dönem projesi ayrıntılı anlatıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- **Akış verisinde giriş hızı sistemin kontrolünde değildir.** İlişkisel veritabanında veriyi geliştirici ekler. Akışta veri dışarıdan, istediği zaman ve hızda gelir.
- **Akış verisi arşivlenir ama sorgulanmaz.** Arşiv genellikle HDFS gibi dağıtık bir sistemdedir. Bu arşiv operasyonel veritabanı değildir; üzerinde sorgu çalıştırmak çok uzun sürer.
- **Asıl hesap sınırlı bellekte yapılır.** Akış hakkındaki özet bilgi (en büyük, en küçük, ortalama, standart sapma vb.) bellekte tutulur ve her yeni elemanla güncellenir.
- **Bloom filtresinde yanlış pozitif olabilir, yanlış negatif olmaz.** Filtre "görmedim" diyorsa %100 doğrudur.
- **İkili sayıya çevirme bilinmeli.** Bir öğrenci sordu. Hoca bunun sınavda bilinmesi gereken temel bir beceri olduğunu söyledi.
- **DGIM'de en fazla hata %50'dir.** Daha fazla bilgi tutularak hata azaltılabilir.
- **Proje için büyük veri kümesi gerekmez.** Amaç MapReduce programı yazabildiğini ve bunu dağıtık ortamda çalıştırabildiğini göstermektir.

Kısa Tekrar Notları

- **Akış verisi:** Sisteme kesintisiz gelen veri. Örnek: sıcaklık, nem ya da sismik sensör verisi. Birden fazla giriş noktasından hızla gelebilir.
- **Anlık sorgu (ad hoc):** Bir kez sorulur. Örnek: "Şu ana kadar görülen en büyük değer ne?" Sistem cevabı bellekte bir değişkende tutar. Soru gelince bu değeri döndürür. İstek-yanıt (request-response) biçimindedir.
- **Sürekli sorgu (standing):** Sürekli çalışır. Örnek: "Her yeni en büyük değeri bildir." Her yeni eleman geldiğinde sorgu uygulanır. Sonuç varsa çıktı üretilir. Yayınla-abone ol (publish-subscribe) biçimindedir.
- **Sınırlı çalışma belleği:** Özet bilgiler ve sürekli sorgular burada tutulur. Fiziksel bellek ya da hızlı bir NoSQL veritabanı olabilir.
- **Kayan pencere:** Pencere eleman sayısıyla (son N eleman) ya da zaman aralığıyla (son 30 saniye) tanımlanır. Yeni eleman girince en eski eleman çıkar.

- **Pencerede ortalama:** Yeni eleman i girip en eski eleman j çıkınca: $\text{ort}_{\text{yeni}} = \text{ort} + \frac{i-j}{N}$.
- **Bloom filtresi:** n bitlik dizi (başta hepsi 0) ve k hash fonksiyonu. Eklemede her hash'in gösterdiği bit 1 yapılır. Sorguda bitlerden biri 0 ise eleman kesin görülmemiştir.
- **Yanlış pozitif olasılığı:** $\left(1 - e^{-km/n}\right)^k$. Burada m eklenen eleman sayısı, n bit sayısı, k hash sayısı.
- **DGIM:** Akış, 1 sayıları 2'nin kuvveti olan kovalara (bucket) bölünür. Her boyuttan en fazla iki kova olur. Tahminde en eski kovanın yarısı alınır.

Detaylı Açıklamalar

Akış veri yönetim sistemi

- Bileşenler:
  - Birden fazla giriş akışı.
  - Anlık sorgu arayüzü ve sürekli sorgu arayüzü.
  - Sınırlı çalışma belleği (limited working storage).
  - Arşiv deposu (archival storage). Genellikle dağıtık bir sistemdir.
- **Anlık ve sürekli sorgu karşılaştırması (en büyük değer örneği):**
  - Akışta sırayla 1, 5, 2, 7 gelsin.
  - Anlık sorguda: Bellekteki "en büyük" değişkeni 1, 5, 5, 7 olarak güncellenir. Çıktı yalnız soru sorulunca verilir.
  - Sürekli sorguda: 1 gelince çıktı, 5 gelince çıktı, 2 gelince çıktı yok, 7 gelince çıktı üretilir.

Uygulamalar

- **Sorgu akışları:** Google, hangi sorguların dünden daha sık arandığını bilmek ister. Her sorgu akışın bir elemanıdır. Her sorgu için günlük sayaç tutulur.
- **Tıklama akışları:** Yahoo, son bir saatte olağandışı sayıda tıklanan sayfaları bilmek ister. Her bağlantı için tıklama sayılır.
- **IP paketleri:** Anahtar (switch) ve yönlendiricilerden geçen paketler izlenir. Özet bilgiler en iyi yönlendirmede ve hizmet engelleme (denial of service) saldırılarını tespit etmede kullanılır.

Kayan pencere

- Örnek: Pencere boyutu 6. Akış harflerden oluşuyor (sayı gibi düşünülebilir).
- Yeni eleman `k` gelince pencerenin en eski elemanı `s` çıkar. Sonra `l` gelince `d` çıkar.
- N belleğe sığmayacak kadar büyükse bütün pencere tutulmaz. Yalnız özet bilgi (ortalama için toplam ve sayı) tutulur ve her yeni elemanla güncellenir.
- Ortalama güncellemesi:
  - İlk N eleman gelene kadar beklenir. Toplam ve sayı hesaplanır.
  - Sonra her yeni eleman i için, çıkan en eski eleman j ise ortalamaya $(i-j)/N$ eklenir.
- Aynı fikir standart sapma gibi başka istatistiklere de uygulanabilir.

Bloom filtresi

- **Problem:** Bir elemanın akışta daha önce görülüp görülmediğini anlamak.
- **Örnek:** Web tarayıcısı (crawler). İndirdiği her sayfadan yeni URL'ler çıkarır ve bir kuyruğa (frontier) ekler. Bir URL'nin daha önce ziyaret edilip edilmediğini bilmezse aynı sayfaları tekrar tekrar indirir.
- **Derste kullanılan örnek:**
  - Bit dizisi uzunluğu 11 (indeksler 0-10).
  - $h_1(x)$: x'in ikili gösteriminde sağdan sayıldığında tek numaralı bitleri alınır, sayıya çevrilir, mod11 alınır.
  - $h_2(x)$: Aynı işlem çift numaralı bitlerle yapılır.
- **Ekleme adımları:**

Eleman	İkili	Tek bitler	h_1	Çift bitler	h_2
25	`11001`	`101` = 5	5	`10` = 2	2
159	`10011111`	`0111` = 7	7	`1011` = 11	0
585	`1001001001`	`01001` = 9	9	`10010` = 18	7

- Üç eklemeden sonra 0, 2, 5, 7 ve 9 numaralı bitler 1 olur. Dizide 5 tane 1 vardır.
- **Sorgu örneği:** 118 = `1110110`. $h_1 = 1110_2 = 14 \bmod 11 = 3$. $h_2 = 101_2 = 5$. 3 numaralı bit 0 olduğu için 118 kesin olarak görülmemiştir.
- **Yanlış pozitif:** Bütün bitler 1 ise filtre "görmüş olabilirim" der. Bu bitleri başka elemanlar 1 yapmış olabilir.
- **Basit tahmin:** Her hash için yanlış pozitif olasılığı yaklaşık "1'lerin oranı"dır. Örnekte $5/11$. İki hash olduğu için yaklaşık $(5/11)^2 \approx 0.21$.

Bloom filtresinde yanlış pozitif olasılığı (dart modeli)

- t: hedef sayısı, yani bit dizisinin uzunluğu.
- d: atılan dart sayısı, yani eleman sayısı × hash fonksiyonu sayısı.
- Bir dartın belirli bir hedefi vurma olasılığı $1/t$. Vurmama olasılığı $1 - 1/t$.
- d dartın hiçbirinin belirli bir hedefi vurmama olasılığı:

$$\left(1 - \frac{1}{t}\right)^d = \left(\left(1 - \frac{1}{t}\right)^t\right)^{d/t} \approx e^{-d/t}$$

- Bu değer dizideki 0'ların oranıdır. 1'lerin oranı $1 - e^{-d/t}$ olur.
- Yanlış pozitif olasılığı: $(1\text{'lerin oranı})^{\text{hash sayısı}}$.
- **Sayısal örnek:**
  - $t = 10^9$ bit, 5 hash fonksiyonu, 10^8 eleman.
  - $d = 5 \times 10^8$, $d/t = 0.5$.
  - 0'ların oranı $e^{-0.5} \approx 0.607$. 1'lerin oranı ≈ 0.393.
  - Yanlış pozitif olasılığı $0.393^5 \approx 0.0094$.

Son k bitteki 1'leri sayma

- **Problem:** 0-1 akışı geliyor. Pencere boyutu N, $k \leq N$. "Son k bitte kaç tane 1 var?" sorusuna hemen cevap vermek gerekiyor.
- **Örnek uygulama:** Spam filtresi. Normal e-posta 0, spam 1. "Son 100.000 e-postada kaç spam var?"
- **Bütün N biti saklamak uygun değil:** Pencere belleğe sığmayabilir. Sığsa bile her soruda saymak çok zaman alır.
- **İlk fikir (bit sayısına göre bloklar):**
  - Akış, boyutu 1, 2, 4, 8, 16 ... bit olan bloklarla özetlenir. Her bloğun içindeki 1 sayısı tutulur.
  - Aynı boyuttan en fazla iki blok olur.
  - Soru sorulunca tamamen pencere içindeki blokların değerleri toplanır.
  - Sınıra denk gelen blokta 1'lerin düzgün dağıldığı varsayılır. Örnek: 16 bitlik blokta 6 tane 1 var ve bloğun 5 biti gerekiyorsa $6 \times 5/16$ eklenir.
  - Sorun: 1'ler bloğun diğer ucunda toplanmışsa tahmin çok yanlış olur.

DGIM algoritması

- Algoritma adını bulan dört araştırmacının (Datar, Gionis, Indyk, Motwani) baş harflerinden alır.
- Her bit için zaman damgası (timestamp) tutulur.
- **Kovalar 1'lere göre kurulur.** Bir kovadaki 1 sayısı (kova boyutu) 2'nin kuvvetidir. Kova 1 ile başlar ve 1 ile biter.
- **Kurallar:**
  - Aynı boyuttan en fazla iki kova olur.

  - Kovalar örtüşmez.
  - Kovalar boyuta göre sıralıdır: eskiye gidildikçe kovalar büyür.
  - Bitiş zamanı N zaman biriminden daha eskiye düşen kova silinir.
- **Yeni bit gelince:**
  - Bit 0 ise hiçbir değişiklik yapılmaz.
  - Bit 1 ise boyutu 1 olan yeni bir kova açılır.
  - Boyutu 1 olan üç kova olursa en eski ikisi boyutu 2 olan tek kovada birleştirilir.
  - Bu, boyutu 2, 4, 8 … olan kovalar için zincirleme tekrarlanır.
- **Sorgu (son k bitteki 1'ler):**
  - Bitişi son k bit içinde kalan kovalar alınır.
  - En eski kova hariç bunların boyutları toplanır.
  - En eski kovanın boyutunun yarısı eklenir.
- **Hata sınırı:** En eski kovanın boyutu 2^r olsun. Daha yeni kovaların toplam boyutu en az $2^r - 1$'dir. En eski kovanın yarısı tahmin edildiği için hata en fazla %50 olur.

Dönem projesi

- **Proje önerisi (proposal):**
  - Bir sayfa.
  - Tek başına mı, bir arkadaşla mı yapılacağı.
  - Kullanılacak veri kümesi.
  - Kilometre taşları (milestones). Örnek: ilk hafta veri bulma, sonraki iki hafta HDFS kurulumu, sonraki haftalar kod yazma.
- **Teknik içerik:**
  - Apache Hadoop ya da Spark kullanılabilir. İkisi de MapReduce programlamaya izin verir.
  - Uygulama mutlaka HDFS üzerinde çalışmalı.
  - Bilgisayarda 3 düğümlü bir HDFS kümesi kurulacak: fiziksel bilgisayar ve VirtualBox ile kurulan 2 sanal makine. Bir düğüm name node, diğer ikisi worker node olacak.
  - Veri kümesi bilgisayara sığabilir. Örnek: 16 GB bellekli bir dizüstü bilgisayarda Kaggle'dan alınmış 1 GB'lık bir veri kümesi.
- **İki bölüm:**
  1. HDFS bölümü: Grafik arayüzle (GUI) yerel dosya sisteminden dosya seçip HDFS'ye yükleme. HDFS'deki dosyaları listeleme.
  2. MapReduce bölümü: Veri kümesi üzerinde MapReduce ile 5 farklı istatistik fonksiyonu. Örnekler: aralık (range), standart sapma, en küçük, en büyük, ortalama.
- Sonra uygulanan fonksiyonların performans analizi istenecek.
- **Teslim ve sunum:**
  - Son derste sunum (demonstration) yapılacak.

Vize ve sonraki ders

- Vize çoktan seçmeli ve online olacak. Muhtemelen üniversitenin online sınav sistemi üzerinden yapılacak.
- Vizeye kadar işlenen bütün konulardan sorumlu olunacak.
- Gelecek hafta örnekleme (sampling) konusu ve bir bölüm daha işlenecek.

Ders sonrası notlar

- **Bloom filtresi hash fonksiyonları.** Hoca bitleri "tek ve çift numaralı bitler" diye tanımladı ama numaralamanın hangi uçtan başladığını söylemedi. Tablodaki değerlerin tutması için bitler sağdan (en düşük anlamlı bitten) 1'den başlayarak numaralanmalıdır. Soldan numaralansaydı 159'un tek bitleri `1011` olurdu. 25 için iki numaralama aynı sonucu verir. 585 için hesap hocanın anlatımında altyazıya geçmedi; tablodaki değerler bu kurala göre hesaplandı.
- **Sorgu kuralı.** Hoca önce "iki bit de 0 ise görülmemiştir" dedi. Doğru kural hemen ardından söylendi: Bitlerden **en az biri** 0 ise eleman görülmemiştir.
- **Yanlış pozitif formülü.** Dart modelindeki $e^{-d/t}$ sabiti Euler sayısıdır ($e \approx 2.718$). Altyazıda "epsilon" diye geçiyor.

- **Sayısal örneğin sonucu.** Hocanın 0.393^5 hesabını söylediği kısım altyazıda bozuk. Sonuç yaklaşık 0.0094, yani yaklaşık %1.
- **DGIM örneğindeki sayılar.** Tahtadaki örnekte blok sayıları altyazıda karışık geçiyor (önce “25 bit”, sonra “23 bit”). Bu yüzden özette yalnız yöntem verildi.
- **DGIM birleştirme örneğindeki tutarsızlık.** Hocanın söylediği ara durumdan (boyutu 1 olan tek kova) sonra yalnız 0 ve 1 gelirse boyutu 1 olan iki kova olur ve birleştirme gerekmez. Anlatılan zincirleme birleştirmenin olması için araya bir 1 daha girmelidir; slayttaki sıra büyük olasılıkla “1, 0, 1” idir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.